

Lecture 2: Costs and Processing

Vertical Slices, MVPs, Crawling, Walking, and Running

University of Chicago

Outline

1. AI Prompting vs AI System Development
2. Developing an AI System
3. AI Resume Scoring System

AI Prompting vs AI System Development

Sometimes, we want to do something.

Like, for example — decide based on a job applicant's resume whether they should be selected for a phone screen.

Some options...

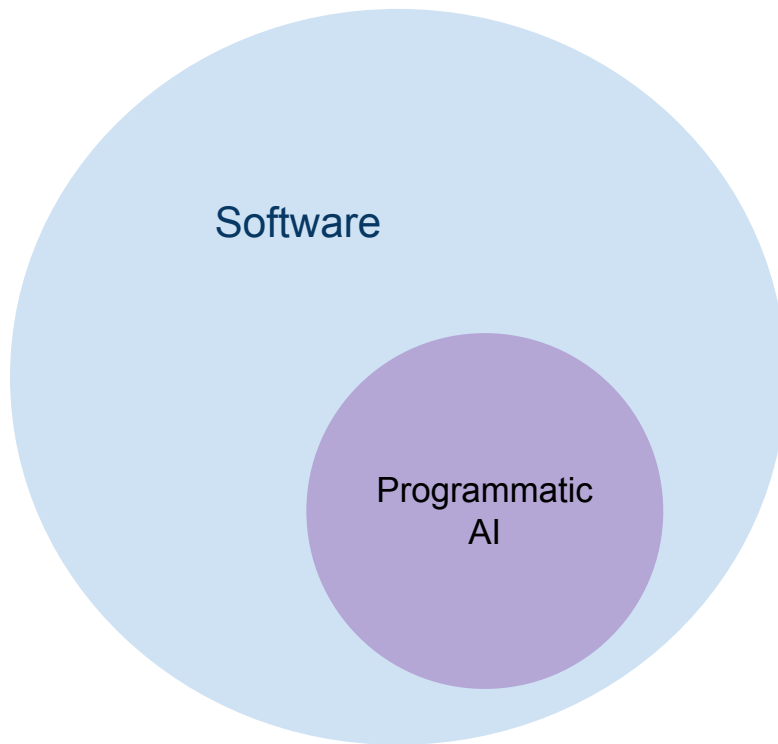
Just read
the resume

Use
software to
filter the
resume

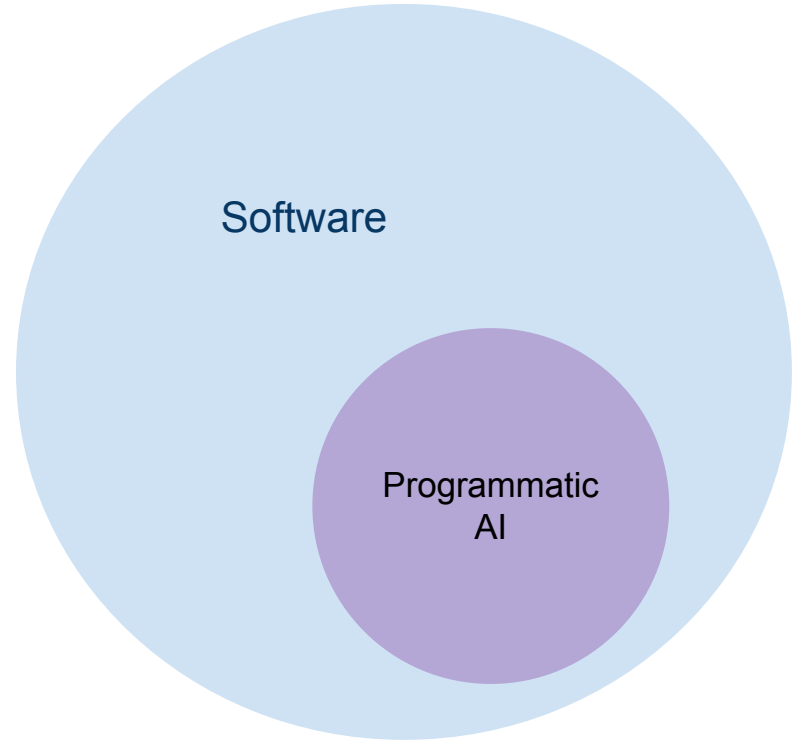
Have an AI
chatbot
“read the
resume”

We said we're focused on "AI Development"

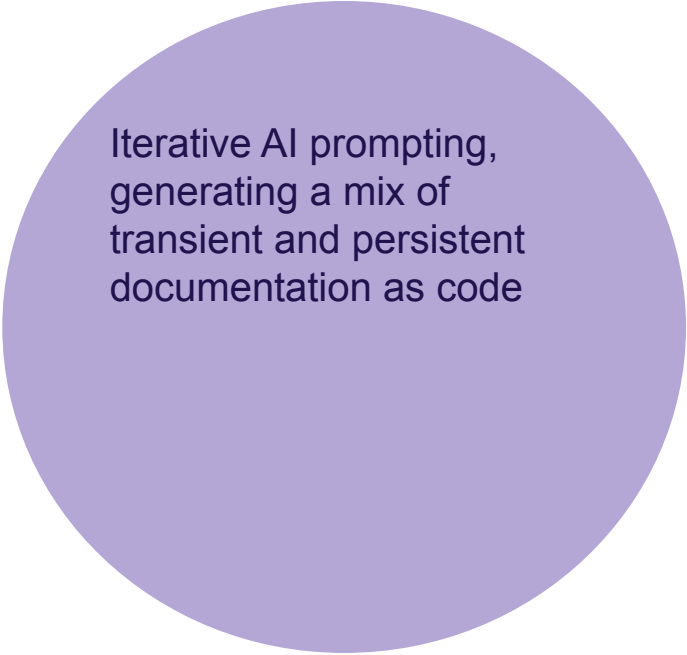
Which I will be referring
to here as "developing
AI systems"



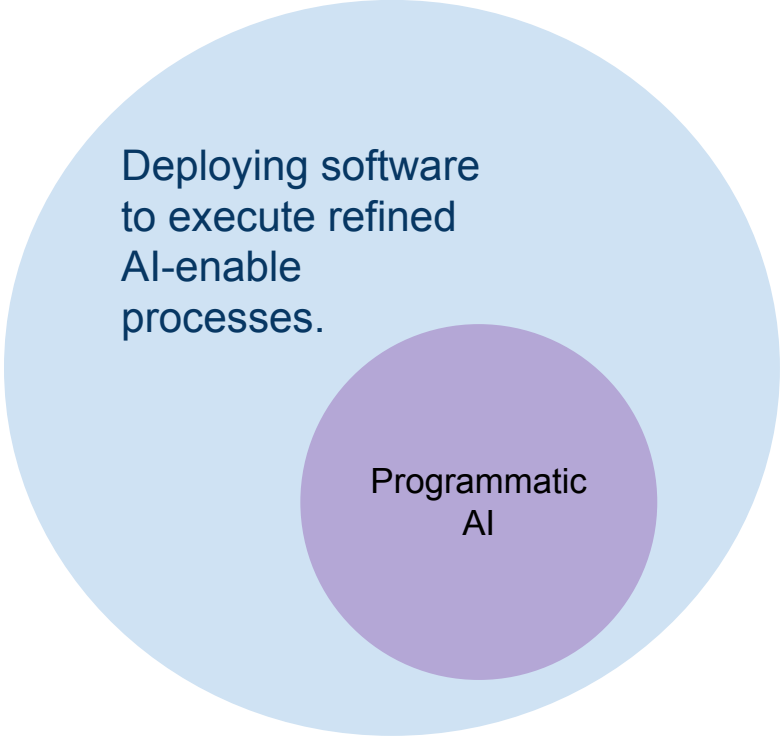
Did we gloss over AI prompting?



Did we gloss over AI prompting?



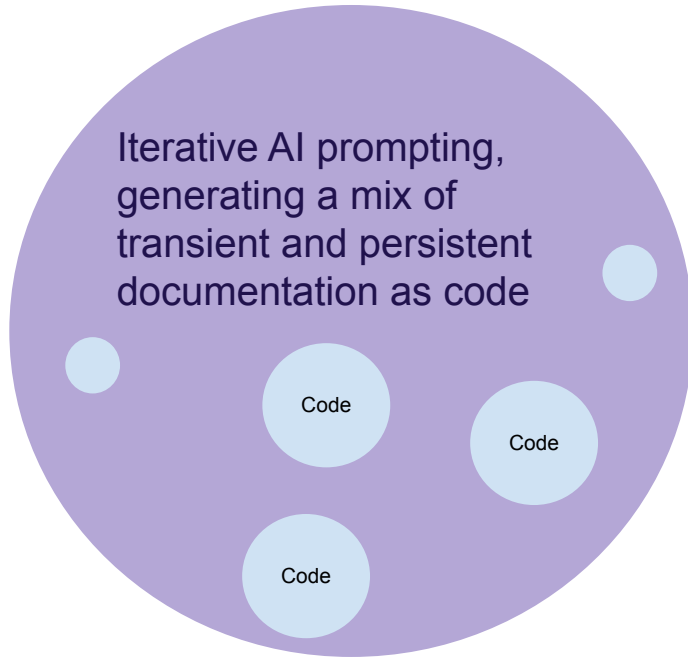
Iterative AI prompting,
generating a mix of
transient and persistent
documentation as code



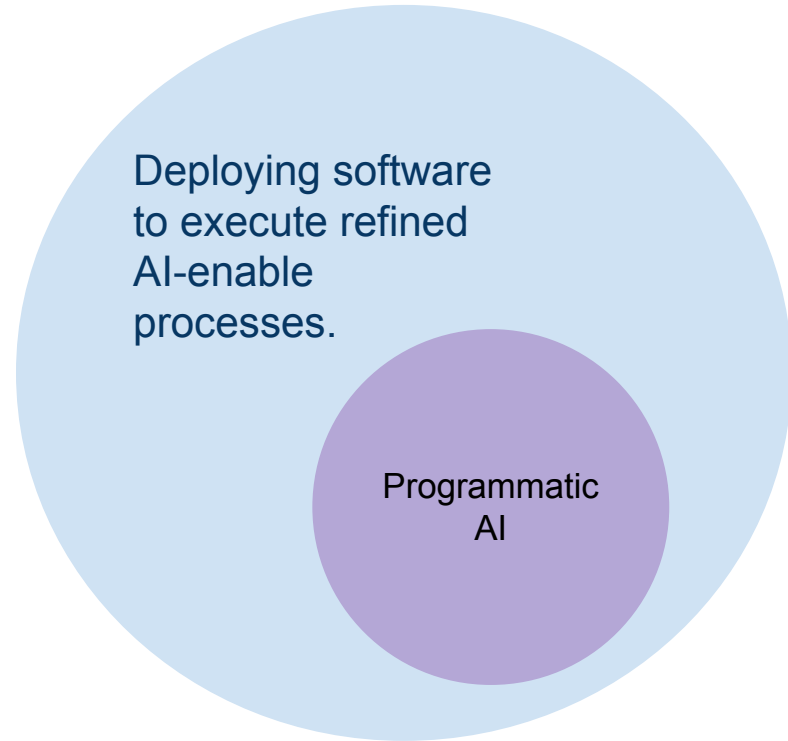
Deploying software
to execute refined
AI-enable
processes.

Programmatic
AI

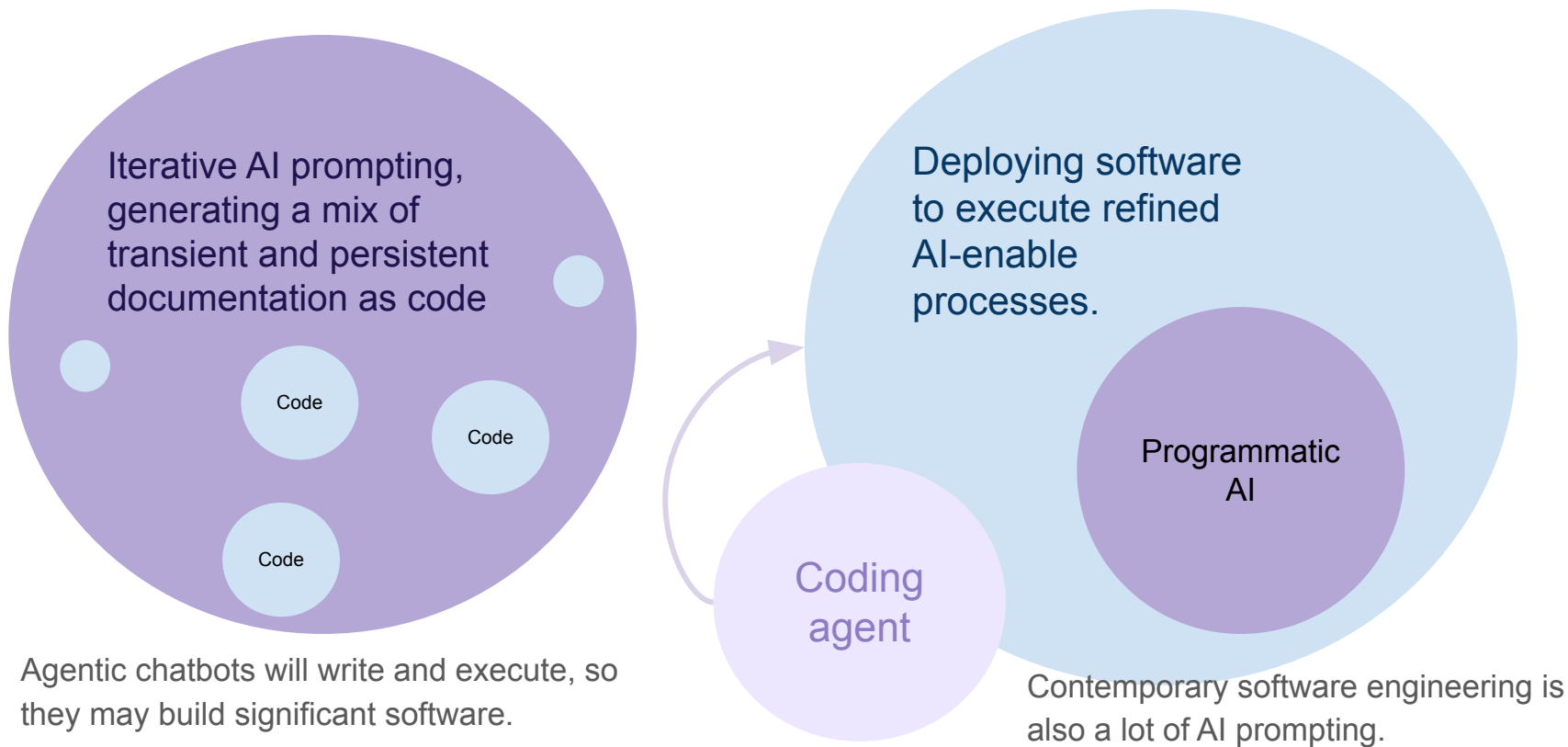
How different are these approaches, really?



Agentic chatbots will write and execute, so they may build significant “software”.



How different are these approaches, really?



AI prompting and developing AI systems they are fundamentally different orientations towards a problem that will yield different results.

As of 2026, software still exists, and **stable, deployable software** is a different kind of output that has several advantages at the cost of being much more difficult to build.

Which is better?

Usually, prompting. For 99% of situations like this, just prompt an AI.

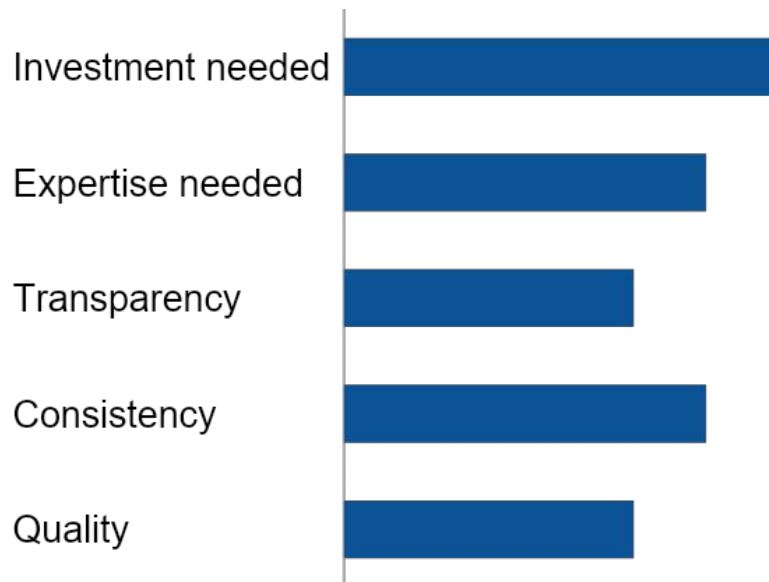
When tackling the types of problems professional AI development teams face? Often it's worth building a real AI system, which enables more consistency, more transparency, and better overall results for repeated tasks that will be performed by many people over time.

Which is better?

AI prompting



AI software systems

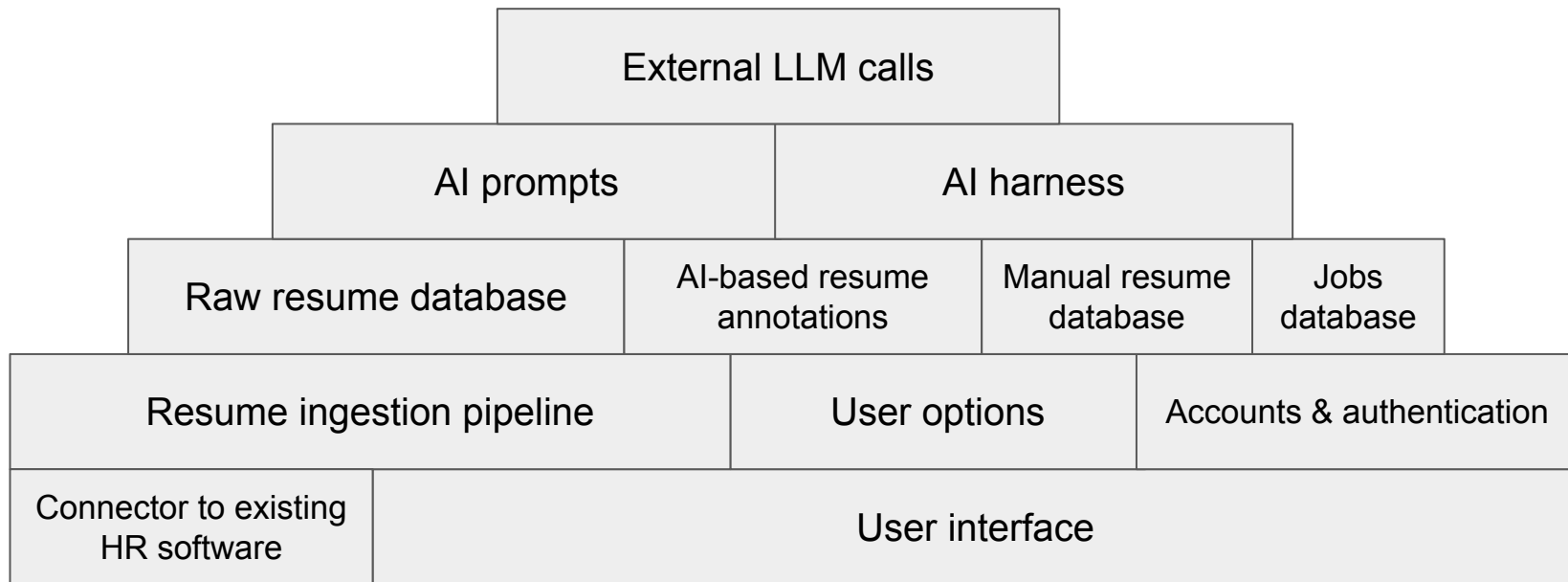


AI prompting is a deep and important skill that is valuable across many roles and industries.

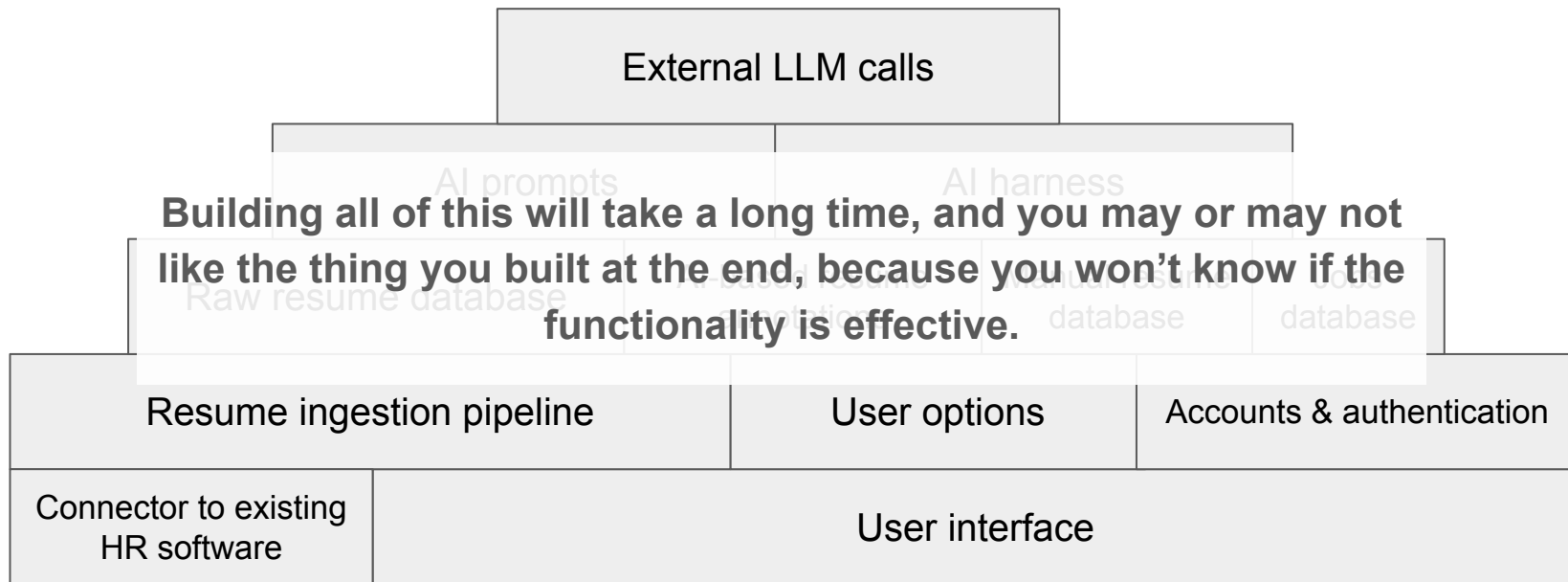
AI system development requires more specialized expertise, is harder to find, and is the focus of this course.

Developing an AI System

What a fully realized system may look like...

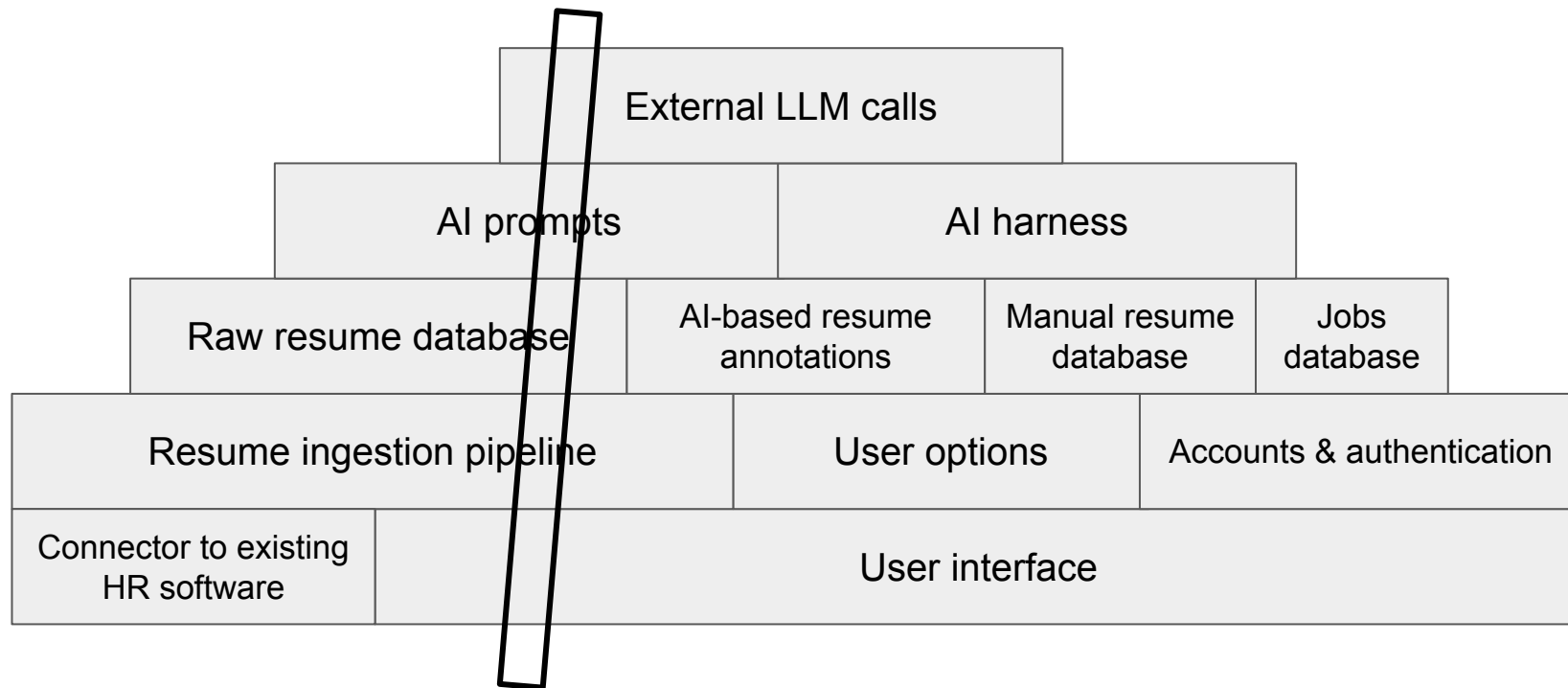


What a fully realized system may look like...



“Agile Development”

Building a vertical slice



What is a Vertical Slice?

A vertical slice:

- Contains **everything** needed to go from top to bottom
- Works on a **subset** of inputs
- Proves the **end-to-end flow** works
- Is the **simplest possible** complete solution

When we build AI systems, we want to start with vertical slices.

The Opposite: Horizontal Development

Horizontal development means:

- Building for **all contingencies** at each level
- Handling **every possibility** before moving forward
- Creating **complete infrastructure** before testing end-to-end

Problem: You spend weeks building infrastructure before knowing if the core idea works!

Example Problem: Time Series Analysis

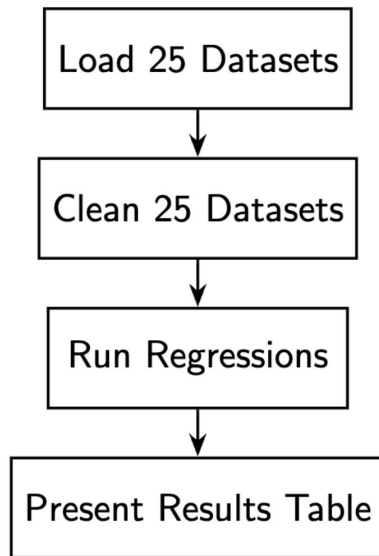
Task: Run a regression on 25 datasets

Context:

- We have data from 25 countries
- Each CSV contains education and GDP data over time
- We want to run a time series analysis on the relationship

Question: How do we break this down?

Time Series Analysis: Complete Problem



This is the **complete problem**, but where do we start?

Time Series Analysis: Complete Problem

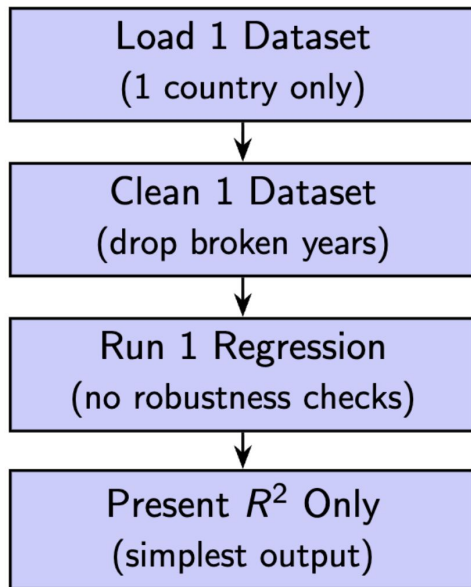
Any of these items could require multiple rounds of iteration & problem solving:

- **Load Data:** What format for each column? How to handle dates? Any transformations?
- **Clean Data:** What cleaning steps? How do we handle missing values?
- **Run Regression:** What type? Linear? Time series?
- **Present Output:** Table? Graph? JSON? What metrics?

This is the complete problem, but we never want to start here!

We want to start with a **vertical slice**.

Time Series Analysis: Vertical Slice



This proves it works end-to-end with minimal effort!

Vertical slices actually enable MORE planning ahead

Before coding, we need to **think systematically**:

1. How do we abstract and organize our code?
2. How do we break tasks into inputs and outputs?
3. How do we organize these into functions?

Answering all of these questions for the whole system is impossible — there are too many unknowns.

But for a vertical slice we can answer each question thoroughly on a reduced problem scope, and then iterate as needed when scope increases.

Time Series Analysis: Comparison

Horizontal development (what not to do)

- Load all 25 datasets with full error handling
- Clean all datasets with every edge case covered
- Build multiple regression types and robustness checks
- Create multiple output formats and visualizations

Vertical slices

- Load one dataset: `load_data()` → `pd.DataFrame`
- Clean one dataset: `clean_data(pd.DataFrame)` → `pd.DataFrame`
- Run one regression: `fit_model(pd.DataFrame)` → `LinregressResult`
- Show R^2 : `get_results(LinregressResult)` → `{"r2": float}`

Why Vertical Slices for AI?

AI systems have unique challenges:

- **Complex**: Many moving parts and dependencies
- **Uncertain failure points**: We aren't sure where things will break
- **Non-deterministic**: Errors appear in unexpected places
- **Iterative prompting**: Prompts need testing and refinement

With vertical slices:

- We build a viable end-to-end solution that works on a **subset** of inputs
- We discover integration issues **early**
- We can demonstrate **value quickly**

The Development Process

The Development Process

Three-Step Approach:

1. Define a vertical slice

- What is the minimal end-to-end flow?
- Start simple, prove it works

2. Define the systems (functions) required

- Know the inputs and outputs of each piece
- Simpler is better

3. Then: Crawl → Walk → Run

- Start minimal, add features incrementally

Example: Personalized Marketing Emails

Task: Send AI-generated personalized marketing emails to leads

Horizontal Approach (what NOT to do):

- Build complete database integration for all leads
- Create email templates for all scenarios
- Build full SMTP sending infrastructure
- Set up monitoring and analytics before sending anything

Vertical Slices:

- Load **one** lead from the database
- Generate **one** email with LLM
- Print email to console for manual review
- If it looks good, move to next step

Principle: Well-Defined Inputs and Outputs

Each component should have:

- **Clear input contract:** What data does it expect? In what format?
- **Clear output contract:** What data does it return? In what format?
- **Single responsibility:** Does one thing well

Benefits:

- Easy to understand what each piece does
- Easy to test each piece independently
- Easy to replace or improve individual pieces
- Clear error boundaries

Methodology: Crawl, Walk, Run

- **Crawl:** What is the minimum code to verify it's possible and establish something to iterate on?
- **Walk:** What is the minimum code to verify it works in most cases and identify possible limitations?
- **Run:** What does the full production system look like?

Stand-Up, In Detail

Goal: Prove the concept works at all

Characteristics:

- Hardcoded examples
- No error handling (let it fail)
- Manual verification
- Don't address edge — or even middle cases
- Evaluate your starting point for the highest risk element in your system
- Proof of concept that it can work in some instances

Stand-Up, In Detail

Goal: Prove the concept works at all

Characteristics:

- Hardcoded examples
- No error handling (let it fail)
- Manual verification
- Don't address edge — or even middle cases
- Evaluate your starting point for the highest risk element in your system
- Proof of concept that it can work in some instances

Example - Resume Screening:

- Hardcode path to one resume PDF
- Load it in a way that works
- Focus on the highest risk element:
 - Extract skills with LLM, print to console
 - Compare to hardcoded job requirements
 - Print match score

Stand-Up, In Detail

Goal: Prove the concept works at all

Characteristics:

- Hardcoded examples
- No error handling (let it fail)
- Manual verification
- Don't address edge — or even middle cases
- Evaluate your starting point for the highest risk element in your system
- Proof of concept that it can work in some instances

Example - Resume Screening:

- Hardcode path to one resume PDF
- Load it in a way that works
- Focus on the highest risk element:
 - Extract skills with LLM, print to console
 - Compare to hardcoded job requirements
 - Print match score

If this doesn't work, nothing else matters!

Methodology: Stand-Up, Walk, Run

Walk: What is the minimum code to verify it works in most cases?

- Handle common edge cases
- Basic error handling
- Works reliably for typical inputs
- Expand scope as needed to reduce risk

Run: What does the full production system look like?

- Comprehensive error handling
- Monitoring and logging
- Scale and performance optimization

Key Takeaways: Building with AI

1. **Define the vertical slice** – what is the minimum thing you can get working?
 - Prove end-to-end flow works with simplest case
 - Identify integration issues early
2. **Figure out the inputs and outputs**
 - Each function needs clear contracts
 - Simpler is better
3. **Figure out the pricing**
 - Know your costs before scaling
4. **Expand horizontally after vertical slice works**
 - Add batch processing, error handling, features
 - Don't optimize or add features prematurely

AI Resume Scoring System

Problem Background

- We have been hired by a company to help with their software developer hiring process
- Our goal is to build an AI-assisted resume scoring system
- For each resume, we want to score it so that we don't waste time on resumes we are unlikely to hire
- We are also cost-conscious.
- Scores should be on a 0-100 scale.

Brainstorming

- How are we going to proceed here?
- What's our vertical slice?
- What are the system components?
- What are the inputs and outputs?
- How much will our decisions cost?

Let's Look at Our Inputs

- **Data:** `resumes_final.csv` has one row per resume (anonymized, taken from Kaggle)
- **Functions:** Our notebook has two functions already:
 1. `load_resumes()` - loads all resumes into a dictionary
 2. `analyze_resume()` - runs a prompt against a resume with structured output
- There is a job req in the directory: `job_req_senior.md`

Define our system, and stand-up!

- Do we have a good definition of each component?
- Do we know the inputs and outputs and what has to happen inside each system?

Next Step: Stand-Up!

Start with the simplest possible version

Generate score for a single resume for a single job req

Walk and run — we'll get here!

Phase 2: Horizontal Expansion (Walk)

- Generate score for **most** resumes for a single job req
- Address variations in resume format, etc

Phase 3: Production (Run)

- Work on **all** resumes for **all** job reqs
- Optimize prompts and costs
- Address edge cases, as needed
- Add caching for repeated skills
- Add human-in-the-loop review
- Monitor accuracy and performance

Next Steps: Hands-On Practice

Goal: Build a working resume scorer

Your Task:

1. **Form teams** (2-3 people)
2. **Open the notebook:** `resume_screening.ipynb`
3. **Work through the examples:**
 - Load resume data
 - Use structured outputs to extract information
 - Match resumes to job requirements
4. **Key questions to answer:**
 - Can we create a score (0-100)?
 - How do we do that?
 - What prompt and schema do we need?
 - How much does it cost per resume?

Focus on the Crawl First – Start simple, prove it works!